Root Finding for Polynomials

David Geddam[1]

[1]Bob Jones University
[1]dgedd236@students.bju.edu

Abstract

Many complex equations require computers and advanced numerical methods to find roots and their behaviors. We seek to find the x roots that satisfy the polynomial equation $P_n(x) = 0$, defined as:

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \cdots + a_1 + a_0$$

where,

- n is degree of the polynomial.
- a_k is real or complex coefficients.

To solve this problem, we employ **Laguerre's method**, which requires evaluation of $P(x)$, $P'(x)$, and $P''(x)$ at each iteration.

Keywords: Laguerre's method; Polynomial;

1 Introduction

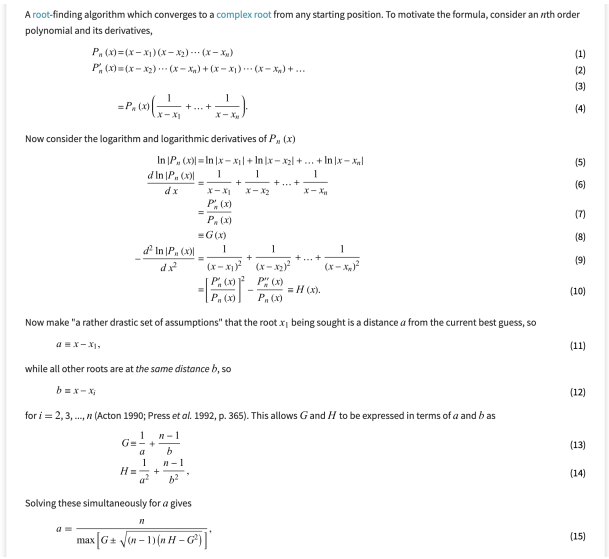
There are many numerical methods today to solve and find roots. Some have limitations and advantages over others. Imagine a graph of a function, like a rollercoaster track. The ground is the x-axis, where $y = 0$. People can find roots for normal and simple equations, but as the coefficients grow and equations become more complex, we need computers and numerical methods for finding roots. Popular methods like Newton's Method can help you find roots, but they effectively assume the ground is flat (a straight line) and take a step based on the slope, while Laguerre proposed a method that assumes the ground is curved (a parabola) and takes a step based on that curve. Since Laguerre assumes the ground is curved, it is usually better at finding the root than Newton. And since Laguerre's method only works for polynomials (e.g., $3x^2 - 2x + 1$) and not $\sin(x)$, e^x, $\log(x)$, etc. Also its advantages, are it converges very fast (fewer steps), and can find complex roots. One more thing to distinguish Newton's method from Laguerre's method is that newton's method looks at a point and draws a tangent down to the ground. Straight lines are bad approximations. To fix this, laguerre's method fits a parabola to the function at our guess point.

2 Method

To solve this, we look at Laguerre's derivation as stated in Wolfram MathWorld. As we know that polynomals involve multiplying terms together, it often gets messy (Product Rule), and addition is much more convenient. Laguerre's method takes the ln of the polynomical first, since logarithms turn multiplication to addition ($\ln(a * b) = \ln(a) + \ln(b)$):

From (8), we state the derivative defined as $G(x)$:

$$G(x) = \frac{P_n'(x)}{P_n(x)} \tag{1}$$



A root-finding algorithm which converges to a complex root from any starting position. To motivate the formula, consider an nth order polynomial and its derivatives,

$$P_n(x) = (x-x_1)(x-x_2)\cdots(x-x_n) \quad (1)$$

$$P_n'(x) = (x-x_2)\cdots(x-x_n) + (x-x_1)\cdots(x-x_n) + \ldots \quad (2)$$

$$= P_n(x)\left(\frac{1}{x-x_1} + \ldots + \frac{1}{x-x_n}\right). \quad (3)(4)$$

Now consider the logarithm and logarithmic derivatives of $P_n(x)$

$$\ln|P_n(x)| = \ln|x-x_1| + \ln|x-x_2| + \ldots + \ln|x-x_n| \quad (5)$$

$$\frac{d\ln|P_n(x)|}{dx} = \frac{1}{x-x_1} + \frac{1}{x-x_2} + \ldots + \frac{1}{x-x_n} \quad (6)$$

$$= \frac{P_n'(x)}{P_n(x)} \quad (7)$$

$$\equiv G(x) \quad (8)$$

$$-\frac{d^2\ln|P_n(x)|}{dx^2} = \frac{1}{(x-x_1)^2} + \frac{1}{(x-x_2)^2} + \ldots + \frac{1}{(x-x_n)^2} \quad (9)$$

$$= \left[\frac{P_n'(x)}{P_n(x)}\right]^2 - \frac{P_n''(x)}{P_n(x)} \equiv H(x). \quad (10)$$

Now make "a rather drastic set of assumptions" that the root x_1 being sought is a distance a from the current best guess, so

$$a \equiv x - x_1, \quad (11)$$

while all other roots are at *the same distance* b, so

$$b \equiv x - x_i \quad (12)$$

for $i = 2, 3, \ldots, n$ (Acton 1990; Press *et al.* 1992, p. 365). This allows G and H to be expressed in terms of a and b as

$$G \equiv \frac{1}{a} + \frac{n-1}{b} \quad (13)$$

$$H \equiv \frac{1}{a^2} + \frac{n-1}{b^2}, \quad (14)$$

Solving these simultaneously for a gives

$$a = \frac{n}{\max\left[G \pm \sqrt{(n-1)(nH-G^2)}\right]}. \quad (15)$$

Figure 1: Laguerre's method derivation (Wolfram MathWorld).

and from (10), it defines $H(x)$ as:

$$H(x) = \left(\frac{P_n'(x)}{P_n(x)}\right)^2 - \frac{P_n''(x)}{P_n(x)} \quad (2)$$

From this, we know $G(x) = \frac{P_n'(x)}{P_n(x)}$, so:

$$H(x) = G(x)^2 - \frac{P_n''(x)}{P_n(x)} \quad (3)$$

After we take a set of assumptions for

$a = x - x_1$ and $b = x - x_i$

we get a (the step size):

$$a = \frac{n}{G(x) \pm \sqrt{(n-1)\big(nH(x) - G(x)^2\big)}} \quad (4)$$

Multiplying top and bottom by $P_n(x)$, we get:

$$a = \frac{nP_n(x)}{P_n'(x) \pm \sqrt{(n-1)\left[nP_n'(x)^2 - nP_n(x)P_n''(x) - P_n'(x)^2\right]}} \quad (5)$$

Inside the square brackets, we can further factor out $P_n'(x)^2$, giving us:

$$a = \frac{nP_n(x)}{P_n'(x) \pm \sqrt{(n-1)\left[(n-1)(P_n'(x)^2) - nP_n(x)P_n''(x)\right]}} \quad (6)$$

and since we know in numerical methods, next guess is current guess minus the step size, we get:

$$x_{k+1} = x_k - \frac{nP_n(x)}{P_n'(x) \pm \sqrt{(n-1)\left[(n-1)P_n'(x)^2 - nP_n(x)P_n''(x)\right]}} \quad (7)$$

Furthermore, we follow this procedure to implement code in Python:

```python
import cmath

def laguerre_step(P, dP, ddP, n, x):
    val = P(x)
    d_val = dP(x)
    dd_val = ddP(x)


    rad = cmath.sqrt((n-1)*((n-1)*d_val**2
    - n*val*dd_val))


    denom1 = d_val + rad
    denom2 = d_val - rad

    if abs(denom1) > abs(denom2):
        chosen_denom = denom1
        return x - (n * val) / chosen_denom
    else:
        chosen_denom = denom2
        return x - (n * val) / chosen_denom
```

3 Experiments

We take a sample test problem,

$$P(x) = x^3 - 6x^2 + 11x - 6 \quad (8)$$

Here, after finding the roots, we got as x = 1, 2, and 3.

Our assumed starting guess, x_0 is zero.

1. Evaluating Derivatives at x_0, we get:

$$P(0) = -6$$
$$P'(0) = 11$$
$$P''(0) = -12$$

2. Calculating step size, a:

$$Radical = \sqrt{(n-1)[(n-1)(P')^2 - nPP'']}$$
$$= \sqrt{2 \cdot [2(121) - 3(72)]} = \sqrt{52} \approx 7.211$$

3. Max Value (applying sign rule)

$$\text{denom1} = |11 + 7.211| = 18.211$$
$$\text{denom2} = |11 - 7.211| = 3.789$$

Since $|18.211| > |3.789|$, we use denom1.

4. Calculating next guess, x_1:

$$x_1 = x_0 - \frac{3(-6)}{18.211} = 0 - (-0.9884) = \mathbf{0.9884}$$

Next, we will write the Python script to automate the next steps:

```python
import cmath

def polynomial(x):
    return x**3 - 6*x**2 + 11*x - 6

def deriv1(x):
    return 3*x**2 - 12*x + 11

def deriv2(x):
    return 6*x - 12

def laguerre_solver(x0, n):
    x = x0
    print(f"\nx = {x0}")
    print(f"{'Iter':<5} | {'Guess (x)':<20} | {'P(x) Value':<20}")
    print("-" * 50)

    for i in range(5):
        val = polynomial(x)
        print(f"{i:<5} | {x.real:.6f} + {x.imag:.1f}i   | {val.real:.6f}")

        # Check if converged
        if abs(val) < 1e-6:
            print(f"Converged to root: {x.real:.4f}")
            break

        # Laguerre Formula
        d_val = deriv1(x)
        dd_val = deriv2(x)

        # Radical
        G = d_val
        rad = cmath.sqrt((n-1)*((n-1)*d_val**2 - n*val*dd_val))

        # Max value
        denom1 = d_val + rad
        denom2 = d_val - rad

        if abs(denom1) > abs(denom2):
            denom = denom1
        else:
            denom = denom2

        a = (n * val) / denom
        x = x - a


# Experiment 1:
laguerre_solver(x0=0, n=3)
```

```
dave@Davids-MacBook-Air ~ % /usr/bin/python3 /Users/d

x = 0
Iter  | Guess (x)            | P(x) Value
--------------------------------------------------
0     | 0.000000 + 0.0i   | -6.000000
1     | 0.988408 + 0.0i   | -0.023589
2     | 1.000000 + 0.0i   | -0.000000
Converged to root: 1.0000
```

4 Limitations

- It requires calculating $P'(x)$ and $P''(x)$. For calculating huge polynomials of massive degrees, this is complex.
- It does not work for non-polynomials $(sin(x), e^x - cos^2(x))$.

5 Conclusion

From our previous single manual step, we moved from 0 to 0.9884. The error is now only 1.1%. And we can check through future experiments. Overall, Laguerre's Method is standard for finding roots in polynomials. While it required complex derivative calculations, its guaranteed convergence and ability to find roots make it better than Newton's method.